Practical Assignment 2 COS330

Hayley Dodkins u21528790

August 2025

Contents

Introduction

Explanation of Multi Factor Authentication Choice

I implemented multi-factor authentication using time-based one-time passwords (TOTP) generated by authenticator apps. This approach was chosen because it does not rely on SMS or email delivery, which can be intercepted or delayed, and instead uses a shared secret between the server and the user's device to generate short-lived codes. The secret is never transmitted after setup, and all subsequent verification depends on cryptographic time-based calculations.

Security is further strengthened by combining TOTP with short token lifetimes and enforced validation through middleware. Tokens are signed, tamper-proof, and tied to user identifiers and roles, which ensures that even if intercepted, they cannot be reused indefinitely. This design provides strong protection against credential theft and replay attacks while remaining straightforward for users to adopt.

Explination of Frontend Framework Choice

Although Ruby on Rails is traditionally used as a full-stack framework, I chose to use it for the frontend. This decision was based on my familiarity with Ruby and the productivity it provides when building user interfaces. Rails' built-in form helpers and integration with ActiveModel made it straightforward to implement strong client-side validation, reducing boilerplate code and ensuring consistent validation rules between the frontend and backend. This allowed me to focus more on application logic and security features while working in an environment I am comfortable and efficient with.

1 Task 1

The following section describes the steps taken to secure the user related endpoints.

1.1 Endpoints and Payloads

1.1.1 Register Endpoint

The register endpoint is used to register a new user. It receives the createUserDTO, as seen in figure 2, which contains the user's first name, last name, email and plain text password. The endpoint will parse the recieved payload using a Zod schema to validate that the payload recieved is the correct format before proceeding. As seen in figure 3, not only does the Zod schema ensure that the payload adheres to the defined schema structure but it will also parse the email to ensure it is a valid email address and check that the password is at

least 8 characters that includes at least one uppercase character, one digit and one special character.

Figure 1: Register endpoint used to register new users.

Figure 2: Register endpoint DTO expected structure.

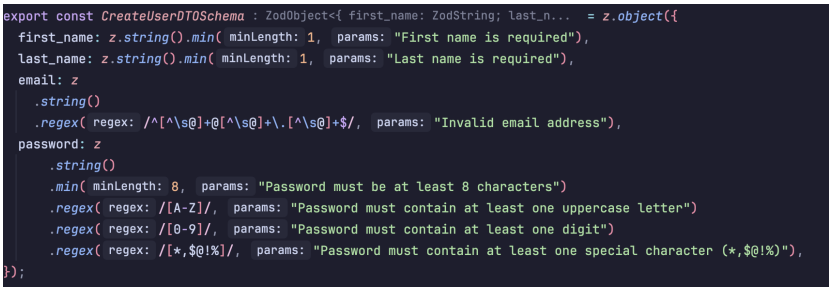


Figure 3: Zod schema to validate register endpoint payloads.

After validating the payload the register endpoint will call the createUser function in the user repository. This function will first validate the email and then it will hash the password. The passwords are hashed using bcrypt through the hashPassword function. Bcrypt includes a randomly generated unique salt for

every password it hashes which is why the application does not manually generate a salt value and store it in the database. By including the salt, Bcrypt insures that identical passwords will generate a different hash value which is how it is resistant to rainbow table attacks. The salting process is followed by multiple rounds of hashing, which slows down the process making it more resistent to brute-force attacks [1].

Once the password is hashed, the generateTotpSecret function is called. This function calls the generateSecret method which will generate a random base32-encoded secret key for each user. This secret is shared between the server and the user's authenticator app. It's the cryptographic seed used to generate time-based one-time passwords. Finally the function builds a Key URI which can be rendered into a QR code on the client side. The user will scan the QR code in an Authenticator app so the app can generate valid 6-digit codes synced with the server.

After generating the secret and url, the user is inserted into the database using the schema shown in figure 4. The is_approved field from figure 5, defaults to false and the role_id defaults to the Guest role as seen in figure 4. This design choice ensures that every new account enters a pending review state, that requires administrative approval. This measure protects against automated account creation and ensures only administrator reviewed accounts can progress beyond the Guest role. Assigning new accounts the least privileged role by default also enforces the principle of least privilege, which minimizes potential damage in the event of an account compromise [2].

```
export async function createUser(
  db: DB,
  dto: CreateUserDTO,
): Promise<MfaResponse> {
  const validated_email = validateAndNormalizeEmail(dto.email);

  if (!validated_email.ok) {
    console.error("Invalid email address");
    return { ok: false, error: "Invalid email address" };
  }
  const email = validated_email.email as Email;

  const validate_psw = validatePassword(dto.password);
  if (!validate_psw.ok) {
    console.error( message: "Invalid password");
    return { ok: false, error: "Invalid password" };
  }

  const psw_hash : string  = await hashPassword(dto.password);

  const uuid : `${string}-${string}-${string}-${string}-...  = randomUUID();
  const { secret, url } = generateTotpSecret(email);
  const guest_role_id : UUID  = await getGuestRoleId(db);

  await db.run(
    sql: `INSERT INTO users (
      user_id, first_name, last_name, email, password_hash,
      created_at, is_approved, sign_in_count, failed_login_attempts,
      role_id, mfa_totp_secret
    ) VALUES (?, ?, ?, ?, ?, unixepoch(), false, 0, 0, ?, ?)`,
    [
      uuid,
      dto.first_name,
      dto.last_name,
      email,
```

Figure 4: createUser function to perform validation logic and insert the user into the database.

```sql
CREATE TABLE IF NOT EXISTS users (
    user_id VARCHAR(36) PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(50) NOT NULL COLLATE NOCASE UNIQUE,
    password_hash TEXT NOT NULL,
    created_at INTEGER NOT NULL DEFAULT (unixepoch()),
    last_login INTEGER DEFAULT NULL,
    is_approved INTEGER NOT NULL DEFAULT 0 CHECK (is_approved IN (0,1)),
    sign_in_count INTEGER NOT NULL DEFAULT 0 CHECK (sign_in_count >= 0),
    failed_login_attempts INTEGER NOT NULL DEFAULT 0 CHECK (failed_login_attempts >= 0),
    current_sign_in_ip VARCHAR(50) DEFAULT NULL,
    last_sign_in_ip VARCHAR(50) DEFAULT NULL,
    role_id VARCHAR(36) NOT NULL REFERENCES roles(role_id) ON UPDATE CASCADE,
    mfa_totp_secret TEXT DEFAULT NULL,
    mfa_enrolled_at INTEGER DEFAULT NULL,
    CHECK (length(user_id) = 36
        AND substr(user_id, 9, 1)  = '-'
        AND substr(user_id, 14, 1) = '-'
        AND substr(user_id, 19, 1) = '-'
        AND substr(user_id, 24, 1) = '-'
        AND substr(user_id, 15, 1) = '4'
        AND lower(substr(user_id, 20, 1)) IN ('8','9','a','b')
    )
);
```

Figure 5: Database schema definition for the user table.

On the client side, the registration form implements validation rules in Ruby using an ActiveModel form object. Each field in the form is bound to this object, which defines a set of validation constraints. These constraints are automatically evaluated when the user submits the form. If any rule is violated, the request does not proceed to the backend API. Instead, the form is re-rendered with error messages. The form validation rules can all be seen in figure 6.

```ruby
class RegisterForm
  include ActiveModel::Model
  include ActiveModel::Attributes

  attribute name :first_name, :string
  attribute name :last_name,  :string
  attribute name :email,      :string
  attribute name :password,   :string

  validates :first_name, presence: { message: "First name is required" }
  validates :last_name,  presence: { message: "Last name is required" }
  validates :email,
            presence: { message: "Email is required" },
            format:   { with: /\A[^\s@]+@[^\s@]+\.[^\s@]+\z/, message: "Please enter a valid email" }

  PASSWORD_REGEX = /\A(?=.*[A-Z])(?=.*\d)(?=.*[*,$@!%]).{8,}\z/

  validates :password,
            presence: { message: "Password is required" },
            format:   { with: PASSWORD_REGEX,
                        message: "Min 8 chars, include 1 uppercase, 1 digit, and 1 special (*,$@!%)" }
end
```

Figure 6: Ruby form model showing client-side validation.

```ruby
 /register
def new_register
  @form = RegisterForm.new
end

hayley-d *
 /register
def create_register
  @form = RegisterForm.new(register_params)

  unless @form.valid?
    flash.now[:alert] = "Please fix the errors below"
    return render :new_register, status: :unprocessable_entity
  end

  conn = Faraday.new(url: BACKEND_BASE_URL, ssl: { verify: false })
  response = conn.post("#{BACKEND_BASE_URL}/register", {
    first_name: register_params[:first_name],
    last_name: register_params[:last_name],
    email: register_params[:email],
    password: register_params[:password]
  })

  body = JSON.parse(response.body)

  if body["user_email"] && body["url"]
    session[:pending_email] = body["user_email"]
    session[:mfa_url] = body["url"]
    redirect_to setup_mfa_path
  else
    flash[:alert] = "Registration failed"
    render :new_register, status: :unprocessable_entity
  end
end
```

Figure 7: Ruby controller method showing client-side validation.

1.1.2 Login Endpoint

The login endpoint is used to authenticate existing users. The endpoint expects a UserLoginDto which is an object that expects an email and passoword. The UserLoginDto is based off of a Zod schema called UserLoginSchema as seen in figure 9. The endpoint will parse the recieved payload against the Zod schema to validate that the payload is the correct format and that the email is a valid email address.

Figure 8: Login endpoint used to authenticate existing users.

Figure 9: Login endpoint used to authenticate existing users.

After ensuring that the received payload is a valid one, the endpoint calls the login function in the user repository. The login function (see figure 10) after validating that the user exists, the user has been approved and MFA for the user is setup will validate that the received plain text password matches the stored password hash. This is done using the `verifyPassword` function (see figure 11) that uses the bcrypt compare function that performs a constant-time comparison to prevent timing attacks [5].

```ts
export async function login(
  db: DB,
  dto: UserLoginDto,
): Promise<RequestUserOption> {
  const user: User | null = await getUserByEmail(db, dto.user_email as Email);

  if (!user) {
    return { ok: false, error: "Invalid login credentials." };
  }

  const user_role : { role_id: string; role_name: string; per...  = await getRoleById(db, user.role_id as UUID);

  if(!user_role || user_role.role_name == "Guest") {
    return { ok: false, error: "User not approved." };
  }

  const isSetup : boolean  = await isMfaSetup(db, user.user_id as UUID);
  if (!isSetup) {
    return { ok: false, error: "MFA not setup" };
  }

  const isValidPassword : boolean  = await verifyPassword(
    dto.password,
    user.password_hash,
  );

  if (!isValidPassword) {
    return { ok: false, error: "Invalid login credentials." };
  }

  return { ok: true, user };
}
```

Figure 10: User repository login function.

```ts
export async function verifyPassword(
  plain: string,
  hash: string,
): Promise<boolean> {
  return bcrypt.compare(plain, hash);
}
```

Figure 11: Function to verify if a plain text password matches the bcrypt hash stored in the database.

On the client side, validation is handled by an ActiveModel form object. This object specifies a set of rules for each field, which are automatically checked when the user submits the form. If any rule is not satisfied, the request is blocked before reaching the backend API, and the form is re-rendered with clear error messages for the affected fields. The complete set of validation rules is shown in figure 13.

```ruby
class LoginForm
  include ActiveModel::Model
  include ActiveModel::Attributes

  attribute :email,      :string
  attribute :password,   :string

  validates :email,
            presence: { message: "Email is required" },
            format:   { with: /\A[^\s@]+@[^\s@]+\.[^\s@]+\z/, message: "Please enter a valid email" }

  PASSWORD_REGEX = /\A(?=.*[A-Z])(?=.*\d)(?=.*[*,$@!%]).{8,}\z/

  validates :password,
            presence: { message: "Password is required" },
            format:   { with: PASSWORD_REGEX,
                        message: "Min 8 chars, include 1 uppercase, 1 digit, and 1 special (*,$@!%)" }
end
```

Figure 12: ActiveModel form object used to validate on the client side.

```ruby
def new
  @form = LoginForm.new
end

hayley-d *
 /login
def login
  @form = LoginForm.new(login_params)

  unless @form.valid?
    flash.now[:alert] = "Please fix the errors below"
    return render :new_register, status: :unprocessable_entity
  end

  conn = Faraday.new(url: BACKEND_BASE_URL, ssl: { verify: false })
  response = conn.post("#{BACKEND_BASE_URL}/login", {
    user_email: login_params[:email],
    password: login_params[:password]
  })

  body = JSON.parse(response.body)

  if body["mfa_required"]
    session[:pending_user] = body["user_id"]
    session[:pending_user_email] = body["user_email"]
    redirect_to otp_path
  else
    flash[:alert] = "Invalid login credentials"
    render :new, status: :unprocessable_entity
  end
end
```

Figure 13: Controller logic showing no API call is made if validation does not pass.

1.1.3 Two Factor Endpoint

The two-factor endpoint is used to register a user for two-factor authentication using the one-time password generated by an authenticator app. First, the endpoint validates that the received payload conforms to the `validateMfaSchema`, as shown in figure 15. It then calls the `validateMfa` function in the user repository as seen in figure 16. This function begins by fetching the user using the submitted email address to ensure the user exists. Next, it takes the stored secret and calculates the expected OTP for the current time window. The expected OTP is compared to the submitted OTP, and if they match, the value is considered valid. Finally, the mfa_enrolled_at field is updated in the database to record that the user has successfully enrolled in two-factor authentication.

Figure 14: Two factor endpoint that will verify that a user submitted the correct OTP.

Figure 15: Schema used to validate the payloads for the two factor endpoint.

Figure 16: Function used to validat if the OTP is valid and update the field in the database.

After updating the user record in the database, a JSON Web Token (JWT) is generated. The tokens are deliberately minimal, containing only essential claims such as user_id, user_email and role_id. It is signed by the server and

configured to expire one hour after issuance, ensuring that credentials cannot be reused indefinitely. By keeping the payload minimal, the design limits the risk of sensitive information being exposed if the token were ever intercepted.

The JWT is delivered to the client as a bearer token, which means it must be included in the `Authorization` header of every subsequent request to protected endpoints. The authentication middleware layer (shown in figure 17) enforces bearer token validation across all protected routes. This middleware verifies both the token's signature and extracts the claims from the token so that downstream handlers can apply access control logic based on the user's role. If the token is missing, expired, or tampered with, the middleware immediately rejects the request.

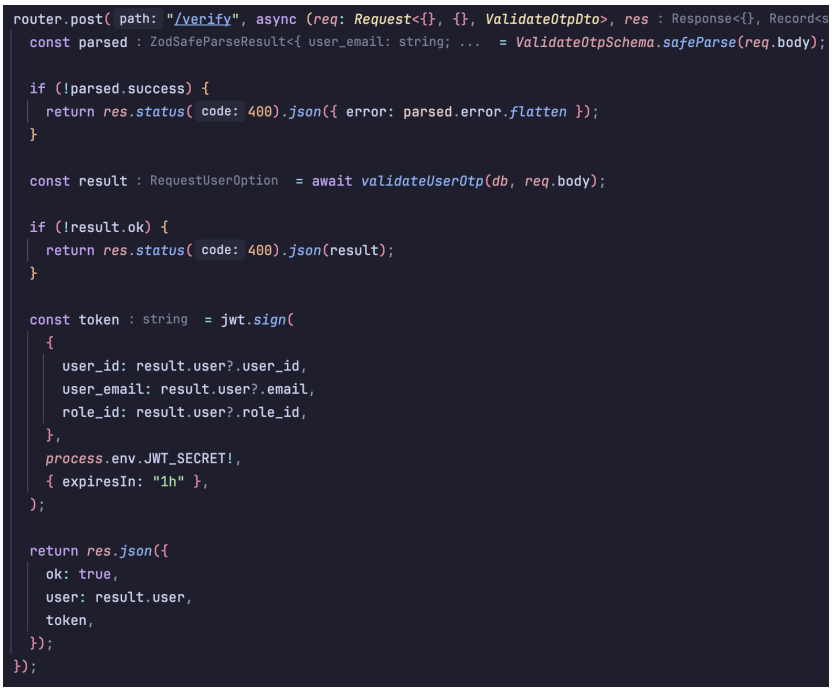
Additionally, when a user attempts to access a restricted endpoint, the bearer token is cross-checked against database records and role permissions. This extra safeguard ensures that expired or revoked accounts cannot continue to access protected resources and helps defend against token replay attacks [4].

```ts
export function authMiddleware(
  req: AuthRequest,
  res: Response,
  next: NextFunction,
) : Response<any, Record<string, any>> | unde...  {
  if (!process.env.JWT_SECRET) {
    throw new Error( message: "JWT_SECRET environment variable not set");
  }

  const authHeader : string | undefined  = req.headers.authorization;
  if (!authHeader || !authHeader.startsWith( searchString: "Bearer ")) {
    return res
      .status( code: 401)
      .json({ error: "Authorization header missing or invalid" });
  }

  const token: string | null = authHeader.split( separator: " ")[1] ?? null;

  if (token === null) {
    return res.status( code: 401).json({ error: "Token is required" });
  }

  try {
    const decoded = jwt.verify(
      token,
      process.env.JWT_SECRET!,
    ) as jwt.JwtPayload & {
      user_id: string;
      user_email: string;
      role_id: string;
    };
    req.user = decoded;
    next();
  } catch (err) {
    return res.status( code: 401).json({ error: "Invalid or expired token" });
  }
```

Figure 17: Authentication middleware used to verify bearer tokens for protected endpoints.

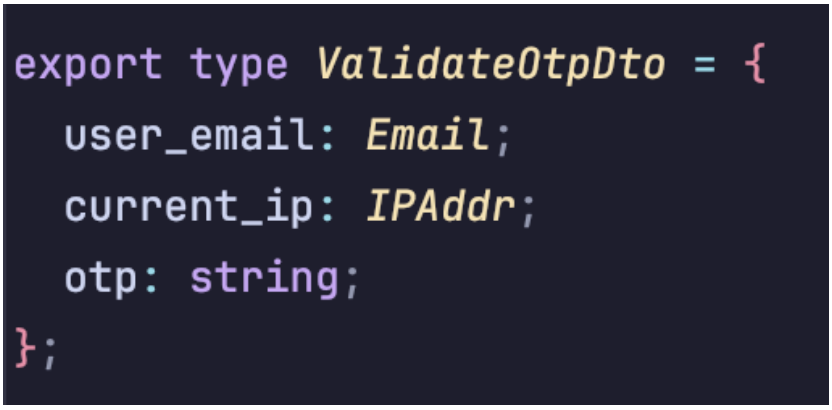
1.1.4 Verify Endpoint

The verify endpoint, shown in figure 18, is used in the second stage of the user login, where they have to submit the authenticator generated OTP. This endpoint will `recievevalidateOtpDto` (seen in figure 19) and validate that the payload matches what is expected using a Zod schema called ValidateOtpSchema (seen in figure 20).



```
router.post( path: "/verify", async (req: Request<{}, {}, ValidateOtpDto>, res : Response<{}, Record<s
  const parsed : ZodSafeParseResult<{ user_email: string; ...  = ValidateOtpSchema.safeParse(req.body);

  if (!parsed.success) {
    return res.status( code: 400).json({ error: parsed.error.flatten });
  }

  const result : RequestUserOption  = await validateUserOtp(db, req.body);

  if (!result.ok) {
    return res.status( code: 400).json(result);
  }

  const token : string  = jwt.sign(
    {
      user_id: result.user?.user_id,
      user_email: result.user?.email,
      role_id: result.user?.role_id,
    },
    process.env.JWT_SECRET!,
    { expiresIn: "1h" },
  );

  return res.json({
    ok: true,
    user: result.user,
    token,
  });
});
```

Figure 18: Verify endpoint that performs the second stage of the user login process.



```
export type ValidateOtpDto = {
  user_email: Email;
  current_ip: IPAddr;
  otp: string;
};
```

Figure 19: Expected DTO for the verify endpoint.

```
export const ValidateOtpSchema : ZodObject<{ user_email: ZodString; curren...  = z.object({
  user_email: z
      .string()
      .regex( regex: /^[^\s@]+@[^\s@]+\.[^\s@]+$/,  params: "Invalid email address"),
  current_ip: z.string().nonempty(),
  otp: z.string().nonempty(),
});
```

Figure 20: Zod schema used to validate the payload for the verify endpoint is correct.

After the payload is checked, the `validateUserOtp` function (shown in figure 21) is used to first ensure a user with the given email exists and then will take the stored secret and calculate the expected OTP for the current time window. The expected OTP is compared to the submitted OTP, and if they match, the value is considered valid.

```
export async function validateUserOtp(
  db: DB,
  dto: ValidateOtpDto,
): Promise<RequestUserOption> {
  const user: User | null = await getUserByEmail(db, dto.user_email);
  if (!user) {
    return { ok: false, error: "User not found." };
  }

  const otp : string  = dto.otp.trim();

  if (user && user.mfa_totp_secret) {
    const ok : boolean  = authenticator.verify({
      token: otp,
      secret: user.mfa_totp_secret,
    });
    if (!ok) {
      await bumpLoginFailure(db, { user_id: user.user_id as UUID });
      return { ok: false, error: "Invalid OTP." };
    }
  }

  await bumpLoginSuccess(db, {
    user_id: user.user_id as UUID,
    current_ip: dto.current_ip,
  });
  return { ok: true, user };
}
```

Figure 21: Function used to validate the submitted OTP.

If the login is successful, the bumpLoginSuccess function (seen in figure 22) is called to update the user record in the database. This resets the failed_login_attempts field to 0, records the new current session IP address, updates the last login timestamp, and increments the sign-in count.

Figure 22: Function to update the user record for every successful login.

As seen in Figure 5, the user schema contains a failed_login_attempts field which tracks the number of consecutive unsuccessful login attempts. If the login process fails, this counter is incremented. The counter enables the system to implement temporary account lockouts and login throttling. This reduces the risk of brute force or credential stuffing attacks by blocking repeated unauthorized access attempts [3].

1.1.5 Manage Endpoint

The manage endpoint, seen in figure 23, is used to approve new users and upgrade them from Guest role to user role. The endpoint is protected so the request will go through the authentication middleware to validate the bearer token before going to the endpoint. The endpoint receives a user Id for the user that is being approved. The endpoint first takes the user ID from the bearer token and ensures that the user exists and has either Admin or Manager role assigned to them. Then the endpoint will call the `approveUser` function, shown in figure 24, which will change the user's role from Guest to User enabling the new user to login.

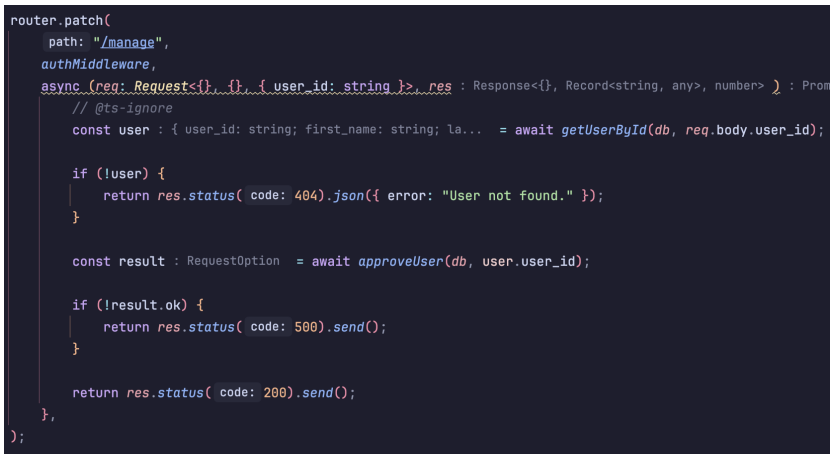


Figure 23: Manage endpoint used to approve new users.

```
export async function approveUser(
  db: DB,
  user_id: string,
): Promise<RequestOption> {
  const role = await getRoleByName(db, "User");
  const user = await getUserById(db, user_id as UUID);

  if (!role || !role.ok) {
    return { ok: false, error: "Role not found." };
  }
  if (!user) {
    return { ok: false, error: "User not found." };
  }

  await db.run("UPDATE users SET role_id = ? WHERE user_id = ?", [
    role.role?.role_id,
    user_id,
  ]);

  return { ok: true };
}
```

Figure 24: Function used to update a user record from guest role to user role.

2 Task 2

To mitigate the risk of man-in-the-middle (MITM) attacks, the server enforces HTTPS for all communications, ensuring that data in transit is encrypted and protected from interception. In addition, every route is secured by requiring a valid bearer token, which provides strong authentication and prevents unauthorized access to protected resources.

2.1 Images

The following are all the endpoints for the images. All image endpoints are protected and require a valid bearer token for access.

2.1.1 Image List

All roles are permitted if the user has a valid bearer token.

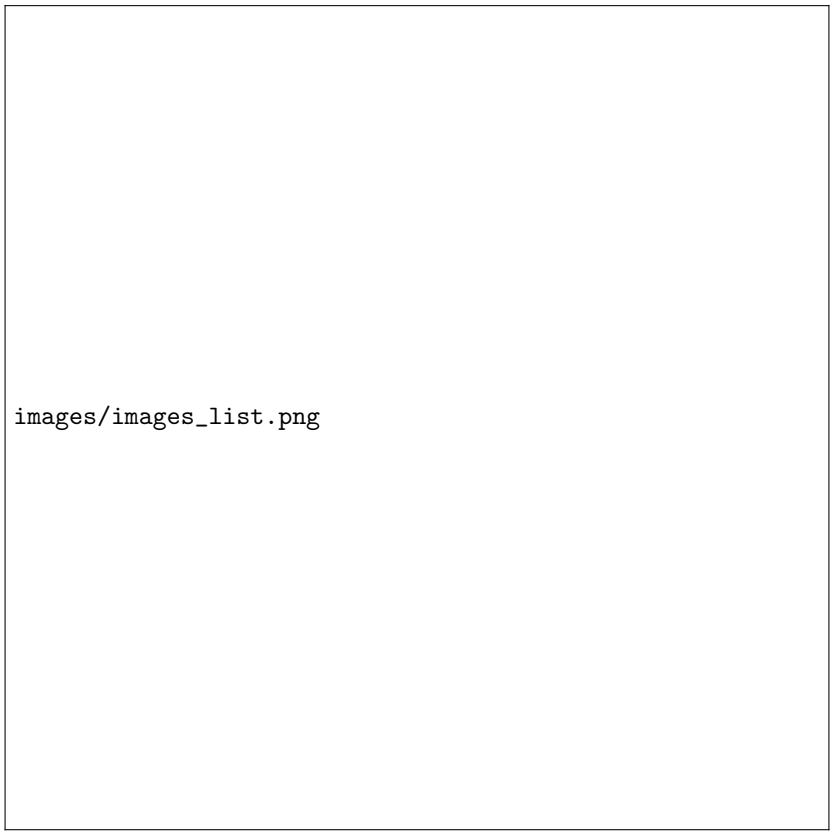


Figure 25: List image documents.

2.1.2 Image Read

All roles are permitted if the user has a valid bearer token.

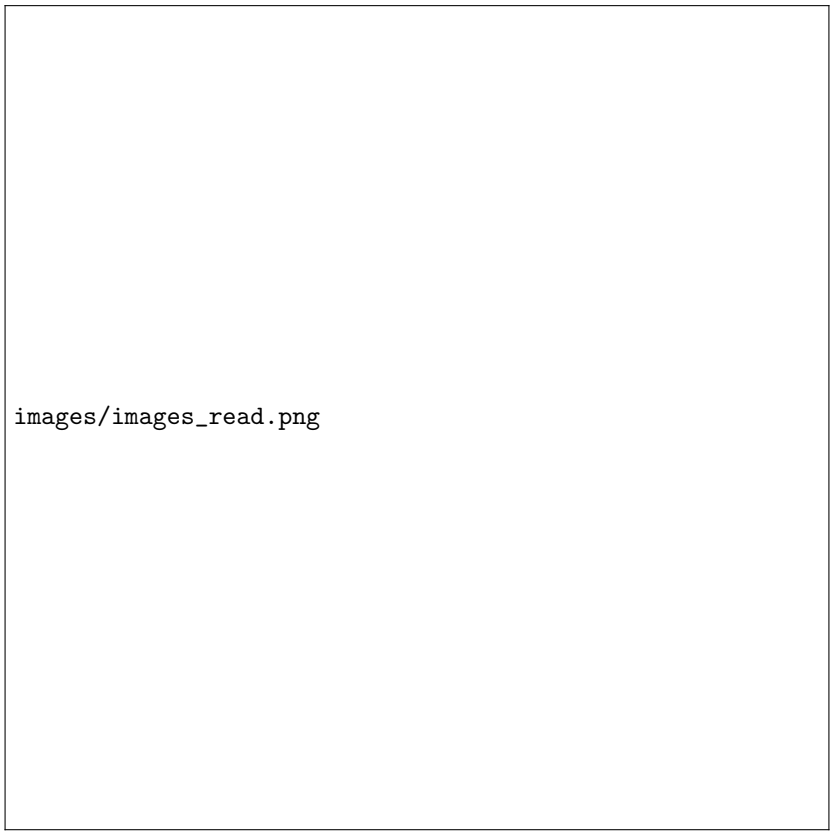


Figure 26: View the details of an image.

2.1.3 Image Create

Users with the guest and user role are denied access, while users with admin and manager roles are permitted to create image documents if the user has a valid bearer token.

Figure 27: Add a new image to the database.

2.1.4 Image Update

Users with the guest role are denied access, while all other roles are permitted to update image information if the user has a valid bearer token.

Figure 28: Updates the details of a given image document.

2.1.5 Image Delete

Users with the guest and user role are denied access, while users with admin and manager roles are permitted to delete image documents if the user has a valid bearer token.

Figure 29: Soft deletes an image from the database.

2.2 Documents

The following are all the endpoints for the documents. All documents endpoints are protected and require a valid bearer token for access.

2.2.1 Document List

Users with the guest role are denied access, while all other roles are permitted if the user has a valid bearer token.

Figure 30: Lists all documents.

2.2.2 Document Read

Users with the guest role are denied access, while all other roles are permitted if the user has a valid bearer token.

Figure 31: Returns a single document with all the metadata.

2.2.3 Document Update

Users with the guest role are denied access, while all other roles are permitted to update documents if the user has a valid bearer token.

Figure 32: Updates the details of a given document.

2.2.4 Document Create

This route is protected and requires a valid bearer token for access. Users with the guest role or user role are denied access, while admin and manager roles are permitted to create documents if the user has a valid bearer token.

Figure 33: Adds a new document to the database.

2.2.5 Document Delete

This route is protected and requires a valid bearer token for access. Users with the guest role or user role are denied access, while admin and manager roles are permitted to delete documents if the user has a valid bearer token.

Figure 34: Removes an existing document from the database.

2.3 Confidential

The following are all the endpoints for the confidential files. All confidential file endpoints are protected and require a valid bearer token for access.

2.3.1 Confidential File List

Users with the guest role are denied access, while all other roles are permitted if the user has a valid bearer token.

Figure 35: List confidential documents.

2.3.2 Confidential File Read

Users with the guest role are denied access, while all other roles are permitted if the user has a valid bearer token.

Figure 36: View the details of a confidential file.

2.3.3 Confidential File Create

Users with the guest role or user role are denied access, while admin and manager roles are permitted to create confidential files if the user has a valid bearer token.

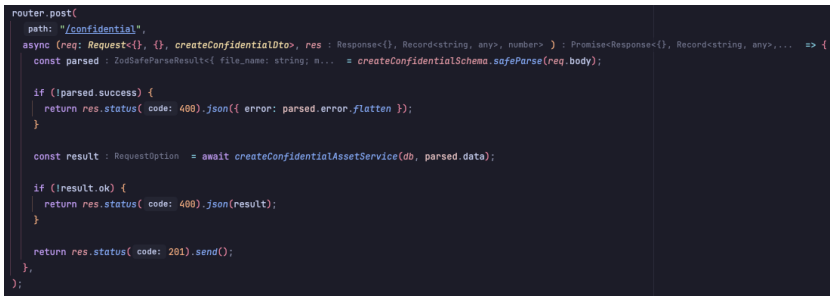


Figure 37: Add a new confidential file to the database.

2.3.4 Confidential File Update

Users with the guest role or user role are denied access, while admin and manager roles are permitted to update confidential files if the user has a valid bearer token.

Figure 38: Updates a confidential file.

2.3.5 Confidential File Delete

Users with the guest role or user role are denied access, while admin and manager roles are permitted to delete confidential files if the user has a valid bearer token.

Figure 39: Soft deletes a confidential file from the database.

3 Task 3

3.1 Impossible Travel Detection Algorithm

The impossible travel detection algorithm works by analyzing login history for each user and looking for movements between IP addresses that would require an unrealistic travel speed. For every user, the system retrieves their request log, ordered by time.

The algorithm compares each login to the next one in sequence. It takes the IP address of the first login and the IP of the following login, and performs a Geo IP lookup on both. This produces approximate latitude and longitude coordinates for each login. Using these coordinates, the algorithm calculates the distance between the two logins with the Haversine formula, which estimates the great-circle distance between two points on Earth.

Next, it computes the time difference between the two logins. Dividing the distance by the elapsed time yields the travel speed. If this calculated speed is greater than 1000 km/h, the login sequence is flagged as an anomaly, since it would imply the user moved faster than is realistically possible between those two locations. For each anomaly, the algorithm records the source IP, destination IP, distance traveled, time difference, estimated speed, and the timestamp of the event.

```
export async function detectImpossibleTravel(db: DB, userId: UUID) : Promise<{ from_ip: string; to_ip: string;...  {
    const history : { origin_ip: string; created_at: number }...  = await db.all<{
        origin_ip: string;
        created_at: number;
    }>( sql: 'SELECT origin_ip, created_at FROM request_log WHERE user_id = ? ORDER BY created_at ASC;', [userId]);

    if (!history || history.length < 2) {
        return [];
    }

    const anomalies: {
        from_ip: string;
        to_ip: string;
        distance_km: number;
        time_diff_minutes: number;
        speed_kmh: number;
        at: number;
    }[] = [];

    for (let i : number  = 0; i < history.length - 1; i++) {
        const a : { origin_ip: string; created_at: number }  = history[i]!;
        const b : { origin_ip: string; created_at: number }  = history[i + 1]!;

        const geoA : Lookup | null  = geoip.lookup(a.origin_ip);
        const geoB : Lookup | null  = geoip.lookup(b.origin_ip);

        if (!geoA || !geoB || !geoA.ll || !geoB.ll) continue;

        const coordsA = geoA.ll as [number, number];
        const coordsB = geoB.ll as [number, number];

        const distanceKm : number  = haversineDistance(coordsA, coordsB);

        const timeDiffHours : number  = (b.created_at - a.created_at) / 3600.0;
        if (timeDiffHours <= 0) continue;

        const speedKmh : number  = distanceKm / timeDiffHours;

        if (speedKmh > 1000) {
            anomalies.push({
                from_ip: a.origin_ip,
                to_ip: b.origin_ip,
                distance_km: Math.round(distanceKm),
                time_diff_minutes: Math.round((b.created_at - a.created_at) / 60),
                speed_kmh: Math.round(speedKmh),
                at: b.created_at
            });
        }
    }

    return anomalies;
```

Figure 40: Determines if a user moved between two locations faster than possible.

3.2 Session Hijacking Detection Algorithm

The session hijacking detection algorithm looks for suspiciously fast changes in a user's IP address, which can indicate that another party has taken over the session. It begins by retrieving the request history for a given user from the database, ordered by timestamp. Each record contains the originating IP address and the time of the request.

The algorithm then examines the history pair by pair, comparing each login

event with the one that immediately follows it. For each pair, it calculates the time difference in seconds. If the IP address changes between the two requests and the time difference is less than 300 seconds, the event is flagged as an anomaly. The reasoning is that a legitimate user cannot usually switch to a completely different network in such a short time window.

```
export async function detectSessionHijacking(db: DB, userId: UUID) : Promise<{ from_ip: string; to_ip: string;...  {
    const history : { origin_ip: string; created_at: number }...  = await db.all<{ origin_ip: string; created_at: number }>(
        sql: `SELECT origin_ip, created_at
     FROM request_log
     WHERE user_id = ?
     ORDER BY created_at ASC`,
        [userId]
    );

    const anomalies: { from_ip: string; to_ip: string; time_diff_seconds: number; at: number }[] = [];

    for (let i : number  = 0; i < history.length - 1; i++) {
        const a : { origin_ip: string; created_at: number }  = history[i]!;
        const b = history[i + 1]!;

        const timeDiffSeconds = b.created_at - a.created_at;

        // If IP changes in < 300 seconds → suspicious
        if (a.origin_ip !== b.origin_ip && timeDiffSeconds < 300) {
            anomalies.push({
                from_ip: a.origin_ip,
                to_ip: b.origin_ip,
                time_diff_seconds: timeDiffSeconds,
                at: b.created_at
            });
        }
    }

    return anomalies;
}
```

Figure 41: Determines if a user moved between two networks faster than possible.

References

[1] Daniel Boterhoven. Why you should use bcrypt to hash passwords. https://danboterhoven.medium.com/why-you-should-use-bcrypt-to-hash-passwords-af330100b861, 2016. Accessed: 23 08 2025.

[2] Cyberark. What is least privilege? https://www.cyberark.com/what-is/least-privilege/. Accessed: 23 08 2025.

[3] Imperva. What is credential stuffing. https://www.imperva.com/learn/application-security/credential-stuffing/#:~:text=Credential%20stuffing%20is%20a%20cyberattack,and%20passwords%20across%20multiple%20services. Accessed: 23 08 2025.

[4] Lakin Mohapatra. Why jwts can be risky. https://lakin-mohapatra.medium.com/why-jwts-can-be-risky-ad367b2f264f, 2024. Accessed: 23 08 2025.

[5] Twingate Team. What is a timing attack? how it works examples. https://www.twingate.com/blog/glossary/timing%20attack, 2024. Accessed: 23 08 2025.